Llama Tránsito: El Alto — Informe técnico H4_T4

Nov 11, 2026 · @Jhonn

Portada

Campo	Dato
Juego	Llama Tránsito: El Alto
Tarea	H4_T4 · Hito 4 · Videojuego Three.js 3D de innovación social
Carrera	Ingeniería de Sistemas · UNIFRANZ, sede El Alto
Materia	Programación Gráfica y Multimedia I
Docente	Ing. Juan Gabriel Lazcano Balanza
Grupo	Capa 8 (número de grupo por completar)
Integrantes	Por completar (3 integrantes)
Repositorio	github.com/JovenSabroso/llama-andina-el-alto
Tecnologías	Three.js 0.187, Vue 3.5, Vite 8, Node.js 24

Eje de impacto y problema social

El juego aborda la **seguridad vial en El Alto** como problema de salud pública: enseña a reconocer las infracciones que ponen en riesgo al peatón. Este eje no figura entre los 5 del Anexo A, por lo que está pendiente de confirmación con el docente; el plan alternativo es el eje 4 (alfabetización digital).

Justificación del eje. El Alto concentra un tráfico intenso de minibuses, trufis y motos en avenidas comerciales como la 16 de Julio. Pasar en rojo, usar el celular al volante, ir con exceso de velocidad o sin casco son conductas cotidianas que el peatón sufre a diario. Un juego permite practicar la observación de esas conductas sin el riesgo de la calle real.

Elemento	Descripción
Problema	Infracciones de tránsito que ponen en peligro a peatones en las avenidas de El Alto

Elemento	Descripción
A quién afecta	Peatones (escolares, comerciantes, cholitas que cruzan a la feria) y los propios conductores
Público del juego	Jóvenes de 12 a 25 años de El Alto, futuros conductores y peatones diarios
Cambio que busca	Que el jugador identifique la infracción y entienda por qué la norma protege a alguien

Mecánica con propósito. La llama solo gana puntos si multa a quien realmente infringe; multar a quien cumple resta 50 puntos y escupir a una ambulancia resta 150. Así el jugador no puede escupir a ciegas: tiene que leer la señal (globo de celular, velocidad, casco) o mirar el semáforo para detectar a quien cruza en rojo. Cada infractor que se escapa bajo la pasarela cuesta una vida, que representa la confianza ciudadana.

Impacto medible. Al terminar, el juego muestra el porcentaje de infracciones detectadas, las multas correctas, las injustas y los infractores que escaparon, junto con un consejo real de seguridad vial. Ese porcentaje permite comparar partidas y medir si el jugador mejora su capacidad de reconocer infracciones.

Assets propios de El Alto. Foto real del Illimani desde El Alto, casas de ladrillo con fierros y tanques en el techo, minibuses con letreros de rutas reales (Ceja, Senkata, Río Seco, 16 de Julio) y bultos de aguayo, cholitas con pollera y sombrero, wiphala, teleférico, pasarela peatonal, baches y propaganda política en fachadas.

Guion del videojuego

Título: Llama Tránsito: El Alto.

Sinopsis (128 palabras). En la avenida 16 de Julio de El Alto, nadie respeta el semáforo. Los minibuses gritan "¡hay campito!", las motos van sin casco y los choferes miran el celular mientras las cholitas intentan cruzar a la feria. Cansada de ver sustos en la cebra, una llama de carga se pone gorra de tránsito, monta un radar de velocidad en su silla y sube a la pasarela peatonal. Desde ahí hace lo que mejor sabe hacer una llama: escupir. Pero escupe con justicia: solo a quien rompe la norma. Si se equivoca, pierde la confianza de los vecinos; si deja escapar a un infractor, un peatón corre peligro. Con ayuda de la Cholita Inspectora, la llama intentará terminar su turno con la avenida más segura.

Personajes

Rol	Personaje	Descripción
Protagonista	La Llama de Tránsito	Llama de carga andina con alforjas verdes, arnés de cuero, gorra de tránsito y radar de velocidad. Escupe a los infractores desde la pasarela.
Aliada	Cholita Inspectora	Aparece cuando la llama acumula 6 multas correctas. Grita "¡YA PUES, MULTA!", frena el tráfico y multa a todos los infractores visibles.
Antagonista (conflicto)	Los conductores infractores	Chofer con celular, minibus que acelera para ganarle al rojo, moto sin casco y auto con exceso de velocidad.
Personajes neutrales	Peatones y cholitas	Cruzan la cebra cuando el semáforo está en rojo y saltan del susto si un infractor pasa cerca.
Trampa	La ambulancia	Puede pasar en rojo legalmente. Escupirle es un error grave.

Estructura narrativa

1. **Planteamiento (Menú).** La cámara recorre la avenida: tráfico, baches, propaganda política y el Illimani al fondo. Se presenta la misión de la llama y las reglas.
2. **Nudo (Juego).** El tráfico crece y se acelera con el tiempo. La llama debe decidir en segundos a quién multar. El semáforo cambia, los peatones cruzan y aparecen ambulancias que ponen a prueba su criterio.
3. **Desenlace (Fin).** Cuando la llama pierde las 3 vidas de confianza ciudadana, termina el turno. La llama baila sobre la pasarela y el juego muestra el porcentaje de infracciones detectadas, un consejo de seguridad vial y un chiste local.

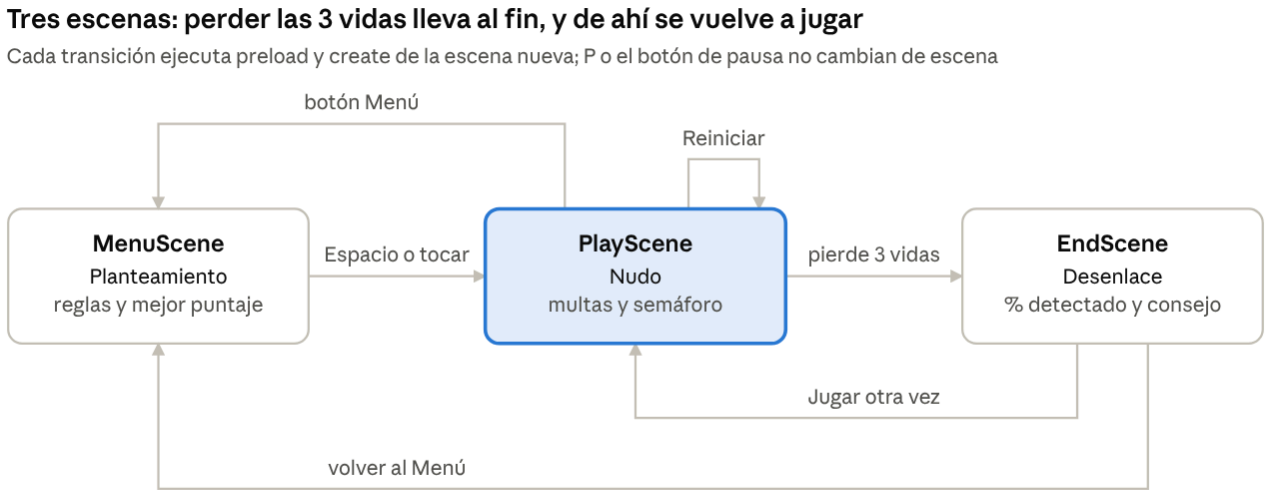
Escena por escena

Escena del diagrama	Momento del guion	Qué ocurre
MenuScene	Planteamiento	Paneo de cámara por la avenida con tráfico de demostración. Panel con título, reglas, controles, mejor puntaje y créditos.

Escena del diagrama	Momento del guion	Qué ocurre
PlayScene	Nudo	La llama cambia de carril, escupe, multa o se equivoca. Semáforo, peatones, baches, ambulancias y el poder de la Cholita.
EndScene	Desenlace	La llama baila, se muestran puntaje, récord, % de detección, consejo y chiste. Opción de jugar otra vez o volver al menú.

Mensaje social final. Las normas de tránsito existen para proteger a quien cruza la calle. Reconocer una infracción es el primer paso para no cometerla: "La llama escupe, pero con justicia".

Diagrama de escenas



flujo de escenas · 3 escenas, 6 transiciones

El jugador entra por el menú, la partida termina al perder la confianza ciudadana y desde el fin puede volver a jugar o al menú; la pausa es un panel de Vue encima de `PlayScene`, no una escena.

Arquitectura y estructura de carpetas

El código separa cuatro capas: Vue (interfaz), el motor `Game` (bucle, cámara, escenas), las escenas (lógica de cada momento) y los módulos de mundo, modelos y sistemas que las escenas reutilizan. Vue solo conoce a `Game`; las escenas nunca tocan el DOM.

```
src/
  main.js, App.vue            arranque de Vue
  components/GameView.vue     monta el juego, HUD, menú, fin, créditos
  assets/img/                 foto del Illimani, texturas y fotos (licencias
libres)
  game/
    Game.js                   renderer, cámara, bucle, resize, cambio de
escenas
    config.js                 constantes del mundo (carriles, semáforo,
pasarela)
    assets.js                 carga de imágenes con caché (fase preload)
    scenes/                   BaseScene, MenuScene, PlayScene, EndScene
    world/World.js            avenida, cebra, semáforo, pasarela, casas,
teleférico
    systems/Traffic.js        vehículos, frenado, colas, infracciones,
peatones
    models/                   Llama, Vehicles, People (modelos 3D por código)
    utils/                    Parallax, textures, input, sfx, storage,
emitter
    data/contenido.js         textos editables: infracciones, consejos, rutas
```

Módulo	Responsabilidad	Por qué separado
`Game.js`	Dueño del renderer, la cámara y el bucle; expone `command`, `start`, `setPaused`	Un solo punto de contacto con Vue
`scenes/*`	Una clase por escena con el ciclo preload → create → update	Requisito R1; cada momento del guion es un módulo ES
`world/World.js`	Escenario estático y semáforo	Las 3 escenas comparten el mismo escenario
`systems/Traffic.js`	Simulación del tráfico y reglas de infracción	Se usa en modo demo (menú, fin) y en modo juego
`models/*`	Llama, vehículos y personas con primitivas de Three.js	Assets propios sin archivos externos
`data/contenido.js`	Textos del juego	El equipo edita textos sin tocar la lógica

Ciclo preload → create → update

Cada escena extiende `BaseScene` y `Game.start(nombre)` ejecuta sus fases en orden: `preload` es asíncrona y el bucle no dibuja la escena hasta que termina; `create` construye los objetos; `update(dt)` se llama en cada frame.

Fase	Qué hace en Llama Tránsito
preload	Descarga la foto del Illimani, las texturas de asfalto y ladrillo y las fotos de los afiches. Usa una caché: cambiar de escena no las vuelve a descargar.
create	Construye el escenario (`World`), el fondo con parallax, el sistema de tráfico y la llama; en `PlayScene` inicializa puntaje, vidas y geometrías del escupitajo.
update	Avanza el semáforo, mueve vehículos y peatones, anima a la llama, integra la física del escupitajo, resuelve colisiones y mueve la cámara.
destroy	Libera geometrías, materiales y texturas propias (no las imágenes en caché).

Cambio de escena en `Game.js` (fragmento real):

```
async start(name, data = {}) {
  const token = ++this._token        // invalida cargas viejas
  const prev = this.current
  this.current = null                // mientras carga no se actualiza ni se
dibuja
  prev?.destroy()                    // libera la escena anterior
  const scene = new SCENES[name](this, data)
  await scene.preload()              // 1. preload (asíncrono)
  if (token !== this._token) return scene.destroy() // se pidió otra escena
mientras cargaba
  scene.create()                     // 2. create
  scene.onResize()
  this.current = scene               // 3. desde aquí el bucle llama a
update(dt)
  this.emit('scene', name)           // Vue cambia de panel
}
```

Bucle principal: el `dt` se limita a 0,05 s para que, al volver de otra pestaña, los vehículos no "salten" a través de la llama.

```
const loop = (timestamp) => {
  this._raf = requestAnimationFrame(loop)
  this.timer.update(timestamp)
  const dt = Math.min(this.timer.getDelta(), 0.05)  // tope de paso
  if (this.current) {
    if (!this.paused) this.current.update(dt)       // la pausa congela la
lógica
    this.renderer.render(this.current.root, this.camera) // pero sigue
dibujando
  }
}
```

Preload con caché (`assets.js`): la promesa se guarda la primera vez y las escenas siguientes reciben las mismas texturas.

```
export function loadAssets() {
  if (!cache) {
    cache = Promise.all([load(illimaniUrl), load(asfaltoUrl, { repeat: true
}), /* ... */])
      .then(([illimani, asfalto, ladrillo, ladrilloNormal, roca, copa]) => ({
/* ... */ }))
  }
  return cache   // segunda escena: sin nuevas descargas
}
```

Efecto 3D con parallax

El juego combina 6 capas a distinta profundidad en el eje Z: dos planos de fondo con un factor de parallax controlado y cuatro capas de geometría 3D real que producen parallax por perspectiva. La cámara es una `PerspectiveCamera` (FOV 55°) ubicada detrás y por encima de la llama, que la sigue con suavizado al cambiar de carril.

#	Capa	Profundidad	Velocidad aparente
1	Foto del Illimani desde El Alto	520 m	p = 0,04 (casi fija)
2	Nubes procedurales	440 m	p = 0,10 + deriva propia
3	Teleférico (cable, torres, cabinas en movimiento)	z = −140	perspectiva real
4	Avenida, casas, baches, propaganda	z = 0 a −235	perspectiva real

#	Capa	Profundidad	Velocidad aparente
5	Vehículos y peatones	z = −175 a +18	se acercan a 13–27 m/s
6	Pasarela, llama y pancarta (primer plano)	z = 0 a +1,3	la más rápida

Por qué un factor `p` en las capas lejanas. Un plano a 500 m casi no se mueve cuando la cámara se desplaza 3 m, así que el parallax físico sería invisible. Se define `p` como "cuánto se mueve la capa comparada con un objeto a 20 m" y se compensa la posición del plano:

```
// utils/Parallax.js — cada frame
update(dt, camera) {
  const c = camera.position
  for (const L of this.layers) {
    const pEff = (L.p * L.dist) / REF_DIST          // REF_DIST = 20 m
    L.mesh.position.set(c.x * (1 - pEff),           // desplazamiento
compensado
                        c.y + L.offY,               // anclado al horizonte
                        c.z - L.dist)               // siempre a la misma
distancia
    if (L.drift) L.map.offset.x = (L.map.offset.x + L.drift * dt) % 1 //
nubes
  }
}
```

Tamaño de cada plano. `layout()` calcula el ancho y alto necesarios a partir del FOV, el aspecto y la inclinación de la cámara, y alinea la base de la cordillera de la foto (58 % desde abajo) con el horizonte. La niebla (`THREE.Fog`, 90–330 m) toma el color promedio de la franja de la ciudad en la foto, para que la avenida 3D se funda con el fondo fotográfico.

Escalamiento a distintas pantallas

El canvas sigue el tamaño de su contenedor con un `ResizeObserver` y, en pantallas verticales, la cámara conserva el campo de visión horizontal para que los 3 carriles siempre entren. Se combinan cuatro decisiones:

1. **Tamaño:** `ResizeObserver` sobre el contenedor (no solo `window.resize`), así responde también a cambios del layout de Vue.
2. **Pixel ratio con tope 2:** en celulares con densidad 3x se ahorra más de la mitad de píxeles sin pérdida visible.

3. **FOV adaptativo:** si el aspecto es menor que 0,8 (celular vertical), se abre el FOV vertical para conservar el horizontal, con un máximo de 100°.
4. **Inclinación adaptativa:** el borde inferior de la pantalla queda siempre a 44° bajo la horizontal; el espacio vertical extra muestra más avenida y cielo en lugar de suelo vacío.

```
// Game.js
resize() {
  const w = Math.max(1, this.container.clientWidth)
  const h = Math.max(1, this.container.clientHeight)
  this.renderer.setPixelRatio(Math.min(window.devicePixelRatio || 1, 2))
  this.renderer.setSize(w, h)
  const aspect = w / h
  this.camera.aspect = aspect
  if (aspect >= MIN_ASPECT) this.camera.fov = BASE_FOV        // 55°
  else {                                                      // vertical:
conservar el FOV horizontal
    const tanH = Math.tan(THREE.MathUtils.degToRad(BASE_FOV / 2)) *
MIN_ASPECT
    this.camera.fov = Math.min(MAX_FOV, THREE.MathUtils.radToDeg(2 *
Math.atan(tanH / aspect)))
  }
  this.camera.updateProjectionMatrix()
  this.current?.onResize()   // la escena recalcula el parallax
}
```

La interfaz de Vue también escala: tamaños con `clamp()`, botones táctiles de al menos 44 px y, bajo 480 px de ancho, la barra de controles se compacta en una sola fila.

Integración Vue ↔ Three.js

El juego vive dentro del componente `GameView.vue`: Vue crea el `Game` al montarse, escucha sus eventos para pintar el HUD y lo destruye al desmontarse. La comunicación es en dos sentidos y pasa siempre por la API de `Game`.

Dirección	Mecanismo	Ejemplos
Juego → Vue	Eventos de un emisor propio (`game.on`)	`score`, `lives`, `combo`, `power`, `light`, `toast`, `best`, `pause`, `scene`, `end`, `loading`
Vue → Juego	Métodos públicos	`game.command('left' / 'right' / 'action' / 'power' / 'pause')`, `game.start('menu')`, `game.restart()`

```
// GameView.vue (fragmento)
onMounted(() => {
  game = new Game(host.value)                    // monta el canvas en el div
del componente
  game.on('score', (v) => (state.score = v))     // el HUD es reactivo
  game.on('light', (v) => (state.light = v))     // semáforo del HUD
  game.on('end', (v) => (state.end = v))         // panel de fin de partida
  game.init()
})

// Limpieza: sin esto cada montaje dejaría un contexto WebGL y un bucle vivos
onBeforeUnmount(() => {
  game?.destroy()
  game = null
})
```

Limpieza al desmontar (`Game.destroy`). Cancela el `requestAnimationFrame`, desconecta el `ResizeObserver`, quita los listeners de teclado y puntero, destruye la escena actual (geometrías, materiales y texturas), libera el renderer con `dispose()` y `forceContextLoss()`, cierra el `AudioContext` y elimina los suscriptores de eventos.

Menú, HUD y fin en Vue, no en Three.js. Los textos se dibujan con HTML encima del canvas: se leen nítidos en cualquier pantalla, son accesibles (botones reales con `aria-label`) y se adaptan con CSS. Las escenas solo emiten datos.

Física, persistencia y decisiones técnicas

Física del escupitajo. Es un proyectil con gravedad (−22 m/s²) integrado en cada frame. Para que sea jugable en pantallas táctiles, el apunte es asistido: se calcula la velocidad inicial que hace que la parábola intercepte al vehículo más cercano del carril, prediciendo dónde estará cuando llegue.

```
// PlayScene.js — apunte asistido
const T = THREE.MathUtils.clamp(dist / 30, 0.3, 0.95)   // tiempo de vuelo
const tz = target.z + target.speed * T                    // posición futura
del vehículo
const ty = target.height * 0.7
vel.set(0, (ty - from.y - 0.5 * GRAVITY * T * T) / T, (tz - from.z) / T)
```

Colisiones. Esfera (radio 0,35 m) contra la caja alineada a los ejes (AABB) de cada vehículo. Si toca el suelo deja una mancha en el asfalto.

Tráfico. Cada vehículo calcula su velocidad segura con $v = \sqrt{(2 \cdot a \cdot d)}$ respecto del obstáculo más cercano: el vehículo de adelante, la línea de pare en rojo o un peatón en la cebra. El infractor "en rojo" ignora el semáforo y acelera, y solo pasa a ser infractor si cruza la línea en rojo. Los baches hacen saltar y frenar al vehículo.

Persistencia (R6). El mejor puntaje se guarda en `localStorage` dentro de `try/catch`, para que el juego siga funcionando en modo incógnito o con el almacenamiento bloqueado.

```
export function saveBest(score) {
  try { localStorage.setItem('llama-andina:best', String(score)) }
  catch { /* sin persistencia: el juego sigue funcionando */ }
}
```

Recuperación al recargar (F5). Además del récord, el juego guarda una "foto" de la partida en `sessionStorage` cada 0,5 s y en el evento `pagehide`: escena, puntaje, vidas, racha, carril, estado del semáforo y cada vehículo con su posición e infracción. Al recargar, `Game.init()` detecta esa foto y restaura la escena en el mismo punto, sin repetir la intro ni pasar por el menú. Se eligió `sessionStorage` y no `localStorage` para que cerrar la pestaña sí empiece una partida nueva.

Decisiones y alternativas comparadas

Decisión	Elegida	Alternativa descartada	Motivo
Modelos 3D	Primitivas de Three.js por código	Modelos .glb descargados	Sin licencias dudosas, sin archivos pesados en git, assets propios y culturales (aguayo, cholitas)

Decisión	Elegida	Alternativa descartada	Motivo
Ventanas, fierros, tanques, postes, baches	`InstancedMesh` (1 llamada de dibujo por tipo)	Un `Mesh` por objeto	Cientos de objetos con pocas llamadas de dibujo
Textura de ladrillo	UV en metros (`boxWorldUV`)	UV por defecto	El ladrillo no se estira en casas de distinto tamaño
Fondo lejano	Plano con foto y factor p	Skybox o geometría lejana	Muestra el Illimani real y el parallax es visible
Sombras	Solo luz direccional, mapa 1024 px	Sombras suaves de alta resolución	Rendimiento en celulares
Pixel ratio	Tope 2	Densidad nativa (hasta 3x)	Hasta 2,25 veces menos píxeles en celulares
Paso de tiempo	`dt` con tope de 0,05 s	`dt` libre	Evita que los vehículos atraviesen la llama tras una pausa
Apunte	Asistido por predicción	Apuntar a mano	Jugable con un solo toque en móvil; la decisión (¿multo o no?) es el reto
Sonido	WebAudio sintetizado	Archivos de audio	Cero descargas y sin problemas de licencia
Textos del juego	`data/contenido.js`	Textos dentro de la lógica	El equipo edita contenido sin romper código

Evidencias de pruebas

El juego se probó en tres tamaños de pantalla con partidas automáticas en Chrome (teclas enviadas por el protocolo DevTools) y no registró errores de consola en ninguna. Las capturas son del 11 de noviembre de 2026, sobre la rama `feat/llama-transito-3d`.

Prueba	Resolución	Qué se verificó	Resultado
Escritorio	1366 × 768	Escena 3D completa, HUD, semáforo, gigantografías, parallax	Correcto, 0 errores
Tablet vertical	820 × 1180	FOV adaptativo, mirada hacia la avenida, controles	Correcto, 0 errores
Celular vertical	390 × 844	3 carriles visibles, controles en una fila, avisos legibles	Correcto, 0 errores
Partida de 60 s	1280 × 720	Multas correctas (celular, en rojo), multas injustas, Cholita	27 avisos de multa registrados
Sin jugar	1280 × 720	Los infractores escapan y restan vidas	2 escapes en 19 s de juego
Build de producción	—	`npm run build`	Sin errores

Nota sobre rendimiento. El navegador de pruebas automáticas no usa GPU y corre a unos 6 FPS, por eso el tiempo de juego avanza a un tercio del real. La fluidez a 60 FPS debe confirmarse jugando en un navegador normal y con las capturas de DevTools en modo dispositivo.

Capturas

Figura 1. Pantalla de inicio: título, reglas y botón de juego (escritorio).



Figura 2. Partida en escritorio, 1366 × 768: HUD, semáforo, gigantografías de alcaldes y parallax con el Illimani.



Figura 3. Tablet vertical, 820 × 1180: el FOV se adapta y la avenida sigue visible.


0
ALCALDE DE EL ALTO
ELISER ROCA
2026 - 2031
UPC
DON PROMESAS CUMPLE (A VECES)
¡BIENVENIDOS A LA 16 DE JULIO!
INTERNET
PELUQUERÍA
¡RESPETA AL PEATÓN!
Cholita Inspectora

Figura 4. Celular vertical, 390 × 844: los 3 carriles caben y los controles se compactan en una fila.



Figura 5. Pantalla de fin de partida con el porcentaje de infracciones detectadas y el consejo de seguridad vial.



Conclusiones e impacto esperado

Llama Tránsito cumple los 7 requisitos técnicos con un juego 3D real, modular e integrado con Vue, y convierte la seguridad vial de El Alto en una mecánica donde el jugador debe reconocer la infracción para ganar.

Req.	Cumplimiento
R1 Escenas modulares	`MenuScene`, `PlayScene`, `EndScene`: una clase por escena en módulos ES
R2 Interacción, física, puntaje	Escupitajo parabólico, colisión esfera–caja, frenado de vehículos, teclado y táctil, marcador y vidas
R3 Parallax 3D	6 capas a distinta profundidad y velocidad en Z
R4 Escalamiento	`ResizeObserver`, pixel ratio ⩽ 2, FOV y mirada adaptativos; probado en 3 tamaños
R5 Vue	Componente `GameView.vue`, eventos y comandos, limpieza al desmontar
R6 Persistencia	Mejor puntaje en `localStorage`
R7 README	Instalación, ejecución, estructura, controles, reglas y créditos

Aprendizajes técnicos. Separar el motor de las escenas permitió reutilizar el mismo escenario en las 3 escenas. Calcular el parallax de las capas lejanas con un factor explícito lo hizo visible y explicable. Limitar el pixel ratio y usar `InstancedMesh` mantuvo cientos de objetos con pocas llamadas de dibujo.

Impacto esperado. El jugador practica, en partidas de 1 a 3 minutos, la observación de cuatro infracciones reales y la regla de prioridad de la ambulancia. El porcentaje de infracciones detectadas que muestra el juego permite medir la mejora entre partidas; una prueba con estudiantes podría comparar ese porcentaje en la primera y la quinta partida.

Mejoras pendientes. Jefe final "El Gran Embotellamiento", confirmar el eje con el docente y probar en celulares físicos de gama baja.

Créditos y fuentes

Recurso	Autor	Licencia
Foto "Nevado del Illimani desde El Alto"	Qhanaaru, Wikimedia Commons	CC0
Textura Asphalt031	ambientCG	CC0
Textura Bricks105	ambientCG	CC0
Foto de Eliser Roca	Asi Es Mi Pueblo	CC BY 2.0
Foto de Eva Copa	Alejandra Vaca, Cámara de Senadores	CC BY 2.0
Modelos 3D, sonidos, letreros, wiphala, aguayo, baches	Grupo	Propios

La propaganda política de partidos y candidatos con lemas es ficticia y en tono de parodia. Los afiches de alcaldes reales solo muestran datos públicos: nombre, cargo, periodo y sigla.

Fuentes de los datos de alcaldes: Eliser Roca (Wikipedia) · Elecciones subnacionales de Bolivia de 2026 (Wikipedia) · 2021 El Alto municipal election (Wikipedia).

Herramientas: Three.js, Vue, Vite, Node.js, VS Code y DevTools del navegador.